8: Ingress

Ingress is all about accessing multiple web applications through a single LoadBalancer Service.

A working knowledge of Kubernetes Services is recommended before reading this chapter. If you don't have this, consider reading the previous chapter first.

The chapter is divided as follows.

- Setting the scene for Ingress
- Ingress architecture
- Hands-on with Ingress

We'll be capitalising the word "Ingress" as it's a resource in the Kubernetes API. This adds clarity and is in-line with recent updates to the official style guide for the Kubernetes docs. We'll also be using the terms "LoadBalancer" and "load-balancer" as follows.

- `LoadBalancer` refers to a Kubernetes Service object of `type=LoadBalancer`
- `load-balancer` is the cloud service performing internet-facing load balancing

As an example… *when you create a Kubernetes LoadBalancer Service, Kubernetes talks to your cloud platform and provisions a cloud load-balancer.*

Finally, the chapter focuses on the latest generally available v1 Ingress as defined in the `networking.k8s.io` API. Earlier beta versions, defined in the `extensions/v1beta1` API, may have some slight variations.

Setting the Scene for Ingress

In the previous chapter, you saw how Service objects provide stable networking for Pods. You also saw how to expose applications to external consumers via NodePort Services and LoadBalancer Services. However, both of these have limitations.

NodePorts only work on high port numbers (30000-32767) and require knowledge of node names or IPs. LoadBalancer Services fix this, but require a 1-to-1 mapping between an internal Service and a cloud load-balancer. This means a cluster with 25 internet-facing apps will need 25 cloud load-balancers, and cloud load-balancers aren't cheap. They may also be a finite resource – you may be limited to how many cloud load-balancers you can provision.

Ingress fixes this by exposing multiple Services through a single cloud load-balancer.

It creates a LoadBalancer Service, on port 80 or 443, and uses *host-based* and *path-based* routing to send traffic to the correct *backend* Service. This is shown in Figure 8.1, and don't worry if it's unclear at this point. We'll keep building the picture, and the hands-on bits should clarify any doubts.

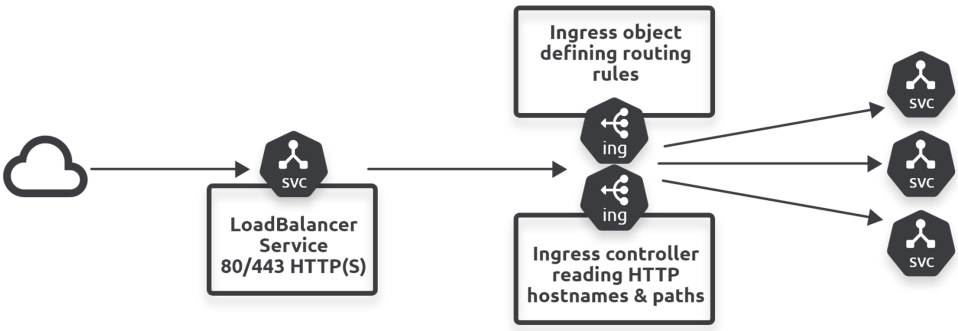

Figure 8.1

Ingress architecture

Ingress is a stable resource in the Kubernetes API. It went GA in Kubernetes 1.19 after being in beta for over 15 releases. During the 3+ years it was in alpha and beta, service meshes increased in popularity and there's some overlap in functionality. As a result, if you plan to run a service mesh, you may not need Ingress.

Ingress is defined in the `networking.k8s.io` API sub-group as a `v1` object and is based on the usual two constructs.

1. A controller
2. An object spec

The object spec defines rules that govern traffic routing, and the controller implements the rules.

However, a lot of Kubernetes clusters don't ship with a built-in Ingress controller – you have to install your own. This is the opposite of other API resources, such as Deployments and ReplicaSets, which have a built-in pre-configured controller. However, some hosted Kubernetes clusters, such as GKE, do pre-install one.

Once you have an *Ingress controller*, you deploy *Ingress objects* with rules that govern how traffic hitting the Ingress is routed.

On the topic of routing, Ingress operates at layer 7 of the OSI model, also known as the "application layer". This means it has awareness of HTTP headers, and can inspect them and forward traffic based on hostnames and paths.

The following table shows how hostnames and paths can route to backend ClusterIP Services.

Host-based example	Path-based example	Backend K8s Service
shield.mcu.com	mcu.com/shield	svc-shield
hydra.mcu.com	mcu.com/hydra	svc-hydra

Figure 8.2 shows two different hostnames (URLs) configured to hit the same load-balancer. An Ingress object is watching, and uses the hostnames in the HTTP headers to route traffic to the appropriate backend Service. This is an example of the HTTP *host-based* routing pattern, and it's almost identical for path-based routing.

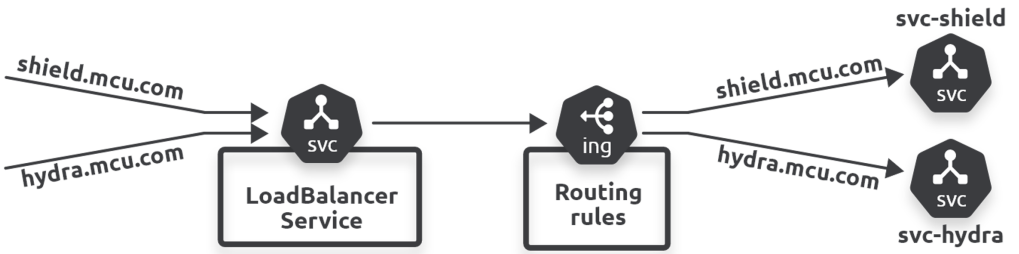

Figure 8.2 Host-based routing

For this to work, name resolution needs to point the appropriate DNS names to the public endpoint of the Ingress load-balancer.

> **Note:** A quick side-step. The "OSI model" is *the* reference model for modern networking. It comprises seven layers numbered 1-7, with the lowest layers concerned with things like signalling and electronics, the middle layers dealing with reliability through things like acks and retries, and the higher layers adding awareness of user apps such as HTTP services. Ingress operates at layer 7, also known as the *application layer*, and implements HTTP intelligence.

In summary, Ingress exposes multiple ClusterIP Services through a single cloud load-balancer. You create and deploy Ingress objects that are rules governing how traffic reaching the load-balancer is routed to backend Services. The Ingress controller, which you usually have to install yourself, uses hostnames and paths to make intelligent routing decisions.

Now that you know the basics, let's see it in action.

Hands-on with Ingress

The following examples require a Kubernetes cluster running on a cloud platform that supports load-balancer integration. All of the major clouds will work. The examples are tweaked for the NGINX Ingress controller, which you'll see how to install, but we'll point out things you'll need to change if you want to run with a different Ingress controller.

If you're following along, you'll need a clone of the book's GitHub repo and you'll need to run all commands from within the `ingress` folder.

```
$ git clone https://github.com/nigelpoulton/TheK8sBook.git
Cloning from...

$ cd ingress
```

Installing the NGINX Ingress controller

It's possible to skip this step if your cluster has a built-in Ingress controller (GKE does).

The NGINX Ingress controller is installed from a YAML file hosted in the Kubernetes GitHub repo. It installs a bunch of Kubernetes constructs including a Namespace, ServiceAccounts, ConfigMap, Roles, RoleBindings, and more.

Install it with the following command. I've split the command over two lines because the URL is so long. If you're following along you'll have to run it on a single line. You should also make sure you're installing the latest release (see `https://github.com/kubernetes/ingress-nginx/releases`).

```
$ kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/
controller-v0.44.0/deploy/static/provider/cloud/deploy.yaml

namespace/ingress-nginx created
serviceaccount/ingress-nginx created
<Snip>
role.rbac.authorization.k8s.io/ingress-nginx-admission created
```

Check the `ingress-nginx` Namespace to make sure the controller Pod is running. It may take a few moments to enter the running phase.

```
$ kubectl get pods -n ingress-nginx \
  -l app.kubernetes.io/name=ingress-nginx

NAME                                        READY   STATUS      RESTARTS   AGE
ingress-nginx-admission-create-brbwh        0/1     Completed   0          27s
ingress-nginx-admission-patch-8dpfl         0/1     Completed   0          27s
ingress-nginx-controller-7fc74cf778-2frpg   1/1     Running     0          33s
```

Don't worry about the `Completed` Pods. These were short-lived to initialise the environment.

Once the controller Pod is running, you've got an NGINX Ingress controller and you're ready to create some Ingress objects. However, before doing that, let's see how to use *Ingress classes* in case your cluster has multiple Ingress controllers.

Configure Ingress classes for clusters with multiple Ingress controllers

If you're following along on a GKE cluster, you'll now have two Ingress controllers.

- The built-in GKE Ingress controller `loadbalancer-controller`
- The NGINX Ingress controller `nginx-ingress-controller`

The way Kubernetes knows which Ingress controller to use when you deploy an Ingress object is via Ingress classes. You create Ingress classes, and then tag Ingress objects with a particular class.

The following YAML object defines an Ingress class called "igc-nginx" for the newly installed NGINX Ingress controller. You can reference this later when deploying Ingress objects.

```
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: igc-nginx
spec:
  controller: nginx.org/ingress-controller
```

Create it with the following command.

```
$ kubectl apply -f ig-class.yml
ingressclass.networking.k8s.io/igc-nginx created
```

You can list and describe it with the usual `kubectl` commands. There is currently no shortname for them, so you'll have to use "ingressclass" when using `kubectl`.

With an Ingress controller and Ingress class in place, you're ready to configure and create Ingress objects.

Configuring host-based and path-based routing

This section deploys two apps and a single Ingress object. The Ingress will be configured to route to both apps via a single load-balancer.

You'll complete all of the following steps.

1. Deploy an app called `shield` and front it with a ClusterIP Service (backend) called `svc-shield`
2. Deploy an app called `hydra` and front it with a ClusterIP Service (backend) called `svc-hydra`
3. Deploy an Ingress object to route the following hostnames and paths
    - Host-based: shield.mcu.com >> svc-shield
    - Host-based: hydra.mcu.com >> svc-hydra
    - Path-based: mcu.com/shield >> svc-shield
    - Path-based: mcu.com/hydra >> svc-hydra
4. A cloud load-balancer will be created and the Ingress controller will monitor it for traffic
5. Configure DNS name resolution to point `shield.mcu.com`, `hydra.mcu.com`, and `mcu.com` to the cloud load-balancer

The overall architecture is shown in Figure 8.3.

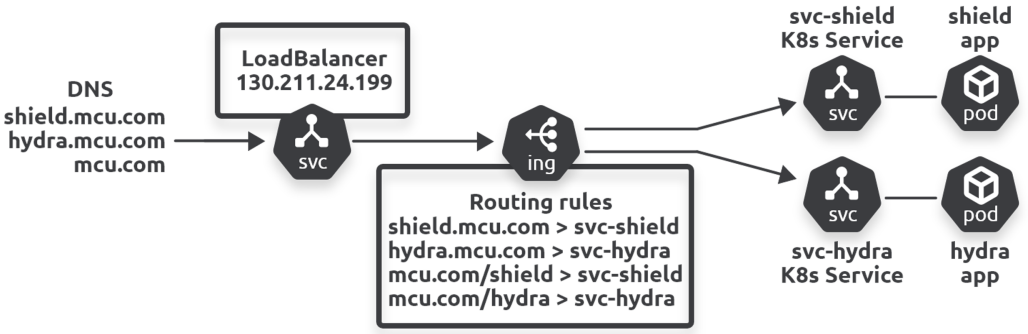

Figure 8.3 Host-based routing

Traffic flow to the `shield` app using host-based routing will be as follows.

1. A client will send traffic to `shield.mcu.com`
2. Name resolution will send the traffic to the load-balancer's public endpoint
3. Ingress will read the HTTP headers for the hostname (`shield.mcu.com`)
4. An Ingress rule will trigger and the traffic will be routed to the `svc-shield` ClusterIP backend
5. The ClusterIP Service will ensure the traffic reaches the `shield` Pod

Deploy the apps

The `shield` and `hydra` apps, as well as associated ClusterIP Services, are defined in the `app.yml` file in the `ingress` folder. It defines the following.

- Two Pods. One called `shield` and one called `hydra`, both listening on port 8080
- Two ClusterIP Services. One called `svc-shield` and the other called `svc-hydra`

Deploy it with the following command.

```
$ kubectl apply -f app.yml
service/svc-shield created
service/svc-hydra created
pod/shield created
pod/hydra created
```

Check them with `kubectl get` and `kubectl describe` commands.

Create the Ingress object

The Ingress object you'll be deploying is defined in `ig-all.yml`. It defines an Ingress object called "mcu-all" with four rules.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mcu-all
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: shield.mcu.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: svc-shield
            port:
              number: 8080
  - host: hydra.mcu.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: svc-hydra
            port:
              number: 8080
  - host: mcu.com
    http:
      paths:
      - path: /shield
        pathType: Prefix
        backend:
          service:
            name: svc-shield
            port:
              number: 8080
      - path: /hydra
        pathType: Prefix
        backend:
          service:
            name: svc-hydra
            port:
              number: 8080
```

Let's step through it.

The first 4 lines define normal stuff, such as the type of object and which schema version to use.

The annotation tells the controller to make a best-effort attempt to re-write paths to the path expected by your

app. This example will re-write incomng paths to "/". For example, traffic hitting the load-balancer via the `/shield` path will have the path re-written to "/". You'll see an example shortly. Also, this annotation is specific to the NGINX Ingress controller and you'll have to comment it out if you're using a different one.

The `spec.ingressClassName` field tells Kubernetes to target this Ingress object at the NGINX controller you installed earlier. You'll have to delete this line, or comment it out, if you're using the built-in Ingress controller.

The rules are a little more complex, so let's look at an example of a host-based rule and a path-based rule.

The following *host-based* rule triggers on traffic arriving via `shield.mcu.com` at the root "/" path and forwards it to a backend Service called `svc-shield`. "svc-shield" is one of the ClusterIP Services deployed earlier via the `app.yml` file.

```
- host: shield.mcu.com           <<=== Traffic arriving via this hostname
  http:
    paths:
    - path: /                    <<=== Arriving at root (no subpath specified)
      pathType: Prefix
      backend:                   <<=== This block references an existing
        service:                 <<=== "backend" ClusterIP Service
          name: svc-shield       <<=== called "svc-shield"
          port:                  <<=== that's listening on
            number: 8080         <<=== port 8080
```

The following *path-based* rule triggers when traffic arrives from `mcu.com/shield`. It gets routed to the same `svc-shield` backend Service.

```
  - host: mcu.com                <<=== Traffic arriving via this hostname
    http:
      paths:
      - path: /shield            <<=== Arriving on this subpath
        pathType: Prefix
        backend:
          service:
            name: svc-shield
            port:
              number: 8080
```

Deploy it with the following command.

```
$ kubectl apply -f ig-all.yml
ingress.networking.k8s.io/mcu-all created
```

Inspecting Ingress objects

List the Ingress objects in the default Namespace. It may take a minute or so for yours to acquire an address. This is while Kubernetes provisions a load-balancer on your cloud.

```
$ kubectl get ing
NAME      CLASS    HOSTS                                  ADDRESS          PORTS
mcu-all   nginx    shield.mcu.com,hydra.mcu.com,mcu.com   130.211.24.199   80
```

The `CLASS` field shows which Ingress class is handling this set of rules. It may show as `<None>` if you only have a single Ingress controller and didn't configure classes. The `HOSTS` field shows it's configured to handle traffic based on three hostnames. The `ADDRESS` column is the public endpoint of the cloud load-balancer. In this example it's an IP address, but some clouds provide a URL. This one is configured for port `80`.

On the topic of ports, Ingress is an HTTP/HTTPS solution.

Describe the Ingress. The output is trimemd to fit the page.

```
$ kubectl describe ing mcu-all
Name:             mcu-all
Namespace:        default
Address:          130.211.24.199
Default backend:  default-http-backend:80   10.120.2.6:8080)
Rules:
  Host            Path      Backends
  ----            ----      --------
  shield.mcu.com  /         shield:8080   10.120.0.19:8080)
  hydra.mcu.com   /         hydra:8080   10.120.0.20:8080)
  mcu.com         /shield   shield:8080   10.120.0.19:8080)
                  /hydra    hydra:8080    10.120.0.20:8080)
Annotations:      nginx.ingress.kubernetes.io/rewrite-target: /
Events:
  Type    Reason  Age          From                      Message
  ----    ------  ----         ----                      -------
  Normal  Sync    85s (x2...)  nginx-ingress-controller  Scheduled for sync
```

Let's step through the output.

The `Address` line is the public IP of the cloud load-balancer associated with the Ingress.

`Default backend` is where the controller sends traffic hitting the Ingress from an unknown host or path. Not all Ingress controllers implement a default backend.

The rules define the mappings between *hosts*, *paths*, and *backends*. Remember that "backends" are usually ClusterIP Services pre-configured to send traffic to application Pods.

The annotations often define integrations with your cloud back-end as well as controller-specific features. This example's telling the controller to re-write all paths to look like they arrived on root "/". This is a kind of *best effort* approach, and as you'll see later, it doesn't work with all apps.

You should also be able to view the Ingress and load-balancer on your cloud backend. Figure 8.4 shows how it looks on the Google Cloud at the time of writing.

Ingress Details REFRESH EDIT DELETE KUBECTL

Type	Ingress
Load balancer	External HTTP(S) LB
IP address	130.211.24.199

Routes

Route	Service	Pod selector	Clusters	Pods
shield.mcu.com/	svc-shield	env=shield	ingress	1/1
hydra.mcu.com/	svc-hydra	env=hydra	ingress	1/1

Serving pods

Service	Name	Status	Restarts	Created on
svc-hydra	hydra	Running	0	Feb 11, 2021, 12:19:45 PM
svc-shield	shield	Running	0	Feb 11, 2021, 12:19:45 PM

Load balancing resources

Load balancer	k8s2-um-8k5xclds-default-mcu-host-hdr21iw2

Figure 8.4 Host-based routing

At this point, the application and backend Services are running and Ingress is configured to route traffic. The only thing left to configure is DNS name resolution so that `shield.mcu.com`, `hydra.mcu.com` and `mcu.com` resolve to the public endpoint of the Ingress.

Configure DNS name resolution

In the real world, you'll configure your internal DNS or internet DNS to point hostnames to the Ingress load-balancer. How you do this varies depending on your environment and who your internet DNS is with.

If you're following along, it's possible to edit the `hosts` file on your laptop or computer as a temporary solution.

On Mac and Linux, edit the `/etc/hosts` file and add the following lines. The example uses the public IP of the Ingress retrieved from the `kubectl get ing` command. Yours will be different.

```
$ sudo vi /etc/hosts

# Host Database
<Snip>
130.211.24.199 shield.mcu.com
130.211.24.199 hydra.mcu.com
130.211.24.199 mcu.com
```

On Windows the file is located at `C:\Windows\System32\drivers\etc\hosts`.

Save your changes.

With this done, any traffic you send to `shield.mcu.com`, `hydra.mcu.com`, or `mcu.com` will be sent to the Ingress load-balancer.

Test the Ingress

Open a web browser and try all of the following URLs.

- shield.mcu.com
- hydra.mcu.com
- mcu.com

Figure 8.5 shows what happens to each request. Notice the traffic for the `mcu.com` request is routed to the *default backend*. This is because there's no ingress rule for `mcu.com`. Depending on your Ingress controller, the message will be different, and you Ingress may not even implement a default backend. The default backend configured by the GKE built-in Ingress returns a helpful message saying "response 404 (backend NotFound), service rules for [/] non-existent ".

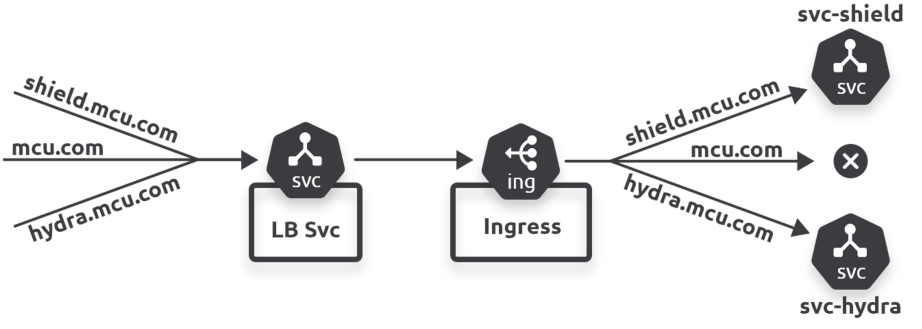

Figure 8.5

Now try connecting to either of the following.

- mcu.com/shield
- mcu.com/hydra

The Ingress uses path-based routing with the *re-write targets* feature to handle these connections. Notice how the image doesn't display for either. This is because not all apps work correctly with path rewrites.

Congratulations, you've successfully configured Ingress for host-based and path-based routing – you've got two applications running on a Kubernetes cluster, fronted by a couple of internal ClusterIP Services, and they're both exposed through a single cloud-based internet-facing load-balancer managed by Kubernetes Ingress!

Clean-up

If you've been following along, you'll have several things running on you Kubernetes cluster. Most importantly, the cloud load-balancer created by the Ingress controller costs money, so be sure to delete it when you're finished.

Delete the Ingress object.

```
$ kubectl delete ingress mcu-all
ingress.networking.k8s.io "mcu-all" deleted
```

Delete the apps and Services using the YAML file. It may take a few seconds for the applications inside the Pods to gracefully terminate.

```
$ kubectl delete -f app.yml
service "svc-shield" deleted
service "svc-hydra" deleted
pod "shield" deleted
pod "hydra" deleted
```

If you want to, you can delete the NGINX Ingress controller with the following three commands.

```
$ kubectl delete namespace ingress-nginx
namespace "ingress-nginx" deleted

$ kubectl delete clusterrole ingress-nginx
clusterrole.rbac.authorization.k8s.io "ingress-nginx" deleted

$ kubectl delete clusterrolebinding ingress-nginx
clusterrolebinding.rbac.authorization.k8s.io "ingress-nginx" deleted
```

Finally, **don't forget to reset your `/etc/hosts` file if you added manual entries earlier**.

```
$ sudo vi /etc/hosts

# Host Database
<Snip>
130.211.24.199 shield.mcu.com   <<=== Delete this entry
130.211.24.199 hydra.mcu.com    <<=== Delete this entry
130.211.24.199 mcu.com          <<=== Delete this entry
```

Chapter summary

In this chapter, you learned that Ingress is a way to expose multiple applications and Kubernetes Services via a single cloud load-balancer. They're stable objects in the API but have feature overlap with a lot of service meshes – if you're running a service mesh you may not need Ingress.

Most Kubernetes clusters require you to install an Ingress controller and lots of options exist. However, some hosted Kubernetes services make things easy by shipping with a built-in Ingress controller.

Once you have an Ingress controller up and running, you create and deploy Ingress objects that are sets of rules governing how incoming traffic is routed to applications and Services on your Kubernetes cluster. It supports host-based and path-based HTTP routing.